

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO (UTEQ)

Facultad de Ciencias de la Computación

Carrera: Software - 5to Semestre "B"

Asignatura: Administración de Base de Datos

Docente: Ing. José Cordero

Período Académico: 2025-2026

INFORME TÉCNICO: PLAN DE RESPALDOS, RECUPERACIÓN Y CONTINUIDAD DE NEGOCIO (ARTISYNC)

Integrantes:

Bone Arroyo Niurca Scarleth

Carvajal Loor Johan Stalin

Figueroa Morales Bryan Javier

Rios Cuyabazo Jhon Kevin

Fecha de entrega: 28 de julio de 2026

Índice

I.	Análisis del proyecto y necesidades de recuperación	2
I-A.	Descripción del Proyecto y Arquitectura de Datos	2
I-B.	Identificación de Datos Críticos	2
I-C.	Volumen y Frecuencia de Cambios	3
I-D.	Objetivos de Recuperación (RPO y RTO)	3
I-E.	Riesgos, Incidentes y Acciones Concretas (Matriz de Mitigación)	3
II.	Diseño y justificación del plan semanal de respaldos	5
II-A.	Matriz de Planificación de Respaldos (7 Días)	5
II-B.	Justificación Técnica del Plan	6
III.	Demostración práctica en pgAdmin y procedimiento de recuperación	7
III-A.	Fase 1: Verificación de la base original	7
III-B.	Fase 2: Creación del Respaldo Completo (Logical Backup)	7
III-C.	Fase 3: Simulación de Desastre (El Error Humano)	7
III-D.	Fase 4: Procedimiento de Restauración	8
IV.	Investigación y simulación Incremental/Diferencial (PostgreSQL WAL/PITR)	9
IV-A.	Fundamentación Teórica	9
IV-A1.	La arquitectura de PostgreSQL y los WAL	9
IV-B.	Simulación de Cadena Incremental y PITR (Consola)	9
IV-B1.	Paso 1: Configuración del Motor (Prerrequisitos)	9
IV-B2.	Paso 2: Generar el Respaldo Base (Full Físico)	9
IV-B3.	Paso 3: Generar la cadena Incremental (Nuevos datos)	9
IV-B4.	Paso 4: Simular Destrucción de la base física	10
IV-B5.	Paso 5: Recuperación Diferencial/Incremental (PITR)	10
V.	Conclusiones y Recomendaciones	11
V-A.	Conclusiones	11
V-B.	Recomendaciones	11
Apéndice		11
A.	Script de Backup Automático (bash)	11
B.	Programación en cron	13
Referencias		13

I Análisis del proyecto y necesidades de recuperación

I-A Descripción del Proyecto y Arquitectura de Datos

ARTISYNC es una plataforma web orientada a la economía creativa, diseñada para conectar creadores de contenido (músicos, diseñadores, programadores) con clientes mediante un modelo de negocio de comisiones y venta de productos digitales. La plataforma busca centralizar el proceso de contratación y venta, permitiendo que tanto la comunicación entre las partes como las transacciones económicas se realicen exclusivamente dentro del entorno web, reduciendo riesgos asociados al uso de canales externos.

El sistema está respaldado por un motor de base de datos **PostgreSQL 18**, estructurado mediante un modelo relacional altamente normalizado que consta de **42 tablas** divididas en 7 módulos principales (basado en la arquitectura detallada en la Entrega 1A):

- **Módulo 1: Seguridad y Control de Acceso (RBAC, JWT, 2FA).**

Tablas: usuarios, roles, permisos, sesiones_usuario, autenticacion_dos_factores, tokens_recuperacion.

- **Módulo 2: Perfiles, Verificación y Portafolio.**

Tablas: perfiles_creadores, estados_verificacion, certificados_ia, habilidades, portafolios, portafolio_items.

- **Módulo 3: Catálogo Dinámico de Servicios.**

Tablas: categorias, subcategorias, servicios, atributos_dinamicos, servicio_atributos, etiquetas.

- **Módulo 4: Motor de Flujos de Trabajo y Pedidos.**

Tablas: flujos_trabajo, etapas_flujo, pedidos, historial_estados_pedido, tickets_revision.

- **Módulo 5: Legal, Entregables y Finanzas (Escrow).**

Tablas: contratos, plantillas_contrato, entregables_finales, pagos_garantia, transacciones_pago.

- **Módulo 6: Comunicación y Notificaciones.**

Tablas: salas_chat, mensajes, documentos_adjuntos, tipos_notificacion, notificaciones_sistema.

- **Módulo 7: Social, Comunidad y Sorteos.**

Tablas: seguidores, comentarios_portafolio, likes_portafolio, resenas_servicios, sorteos, participantes_sorteo.

I-B Identificación de Datos Críticos

No todos los datos tienen el mismo impacto en caso de pérdida. Se ha realizado una clasificación de criticidad basada en el modelo de negocio de ARTISYNC:

■ **Nivel 1 - Criticidad Extrema (Financiero y Legal):**

Tablas `pagos_garantia`, `transacciones_pago`, `contratos`, `pedidos`. Una pérdida en estas tablas implica una discrepancia financiera (patrón Escrow) o la invalidez de un acuerdo legal con firma electrónica. La integridad de estos datos es fundamental para la operación de la plataforma.

■ **Nivel 2 - Criticidad Alta (Operativo y Comunicación):**

Tablas `mensajes`, `salas_chat`, `entregables_finales`. Representan el mayor volumen transaccional de I/O (Input/Output). Perder estos datos afectaría la resolución de disputas entre cliente y creador, además de la trazabilidad de los proyectos.

■ **Nivel 3 - Criticidad Media (Catálogo y Perfiles):**

Tablas `servicios`, `portafolios`, `perfiles_creadores`. Su alteración es molesta pero recuperable mediante intervención del usuario o mediante la recarga de información desde respaldos lógicos.

I-C Volumen y Frecuencia de Cambios

Basado en una proyección de 500 creadores activos y 2000 clientes para la fase inicial de ARTISYNC:

- **Transaccionalidad Financiera:** ~150 transacciones de pedidos/pagos diarios (Frecuencia media, volumen bajo, alta criticidad).
- **Transaccionalidad de Mensajería:** ~10,000 inserciones diarias en la tabla `mensajes` vía WebSockets (Frecuencia extremadamente alta, I/O intensivo).
- **Archivos Binarios (Entregables):** No se almacenan en la BD (se usa AWS S3/Cloudflare R2), pero las URLs referenciales en la BD cambian constantemente, lo que requiere respaldo de los metadatos.

I-D Objetivos de Recuperación (RPO y RTO)

Para garantizar la continuidad del negocio de ARTISYNC, se han establecido las siguientes métricas estrictas:

■ **RPO (Recovery Point Objective - Punto de Recuperación): 15 Minutos.**

- *Definición:* Es la cantidad máxima de datos que la empresa está dispuesta a perder.
- *Justificación:* Debido al manejo de dinero en garantía (Escrow), perder más de 15 minutos de transacciones generaría inconsistencias irreversibles con la API de PayPal. Se logra configurando el archivado de `WAL` cada 15 minutos o cada 16MB.

■ **RTO (Recovery Time Objective - Tiempo de Recuperación): 2 Horas.**

- *Definición:* Es el tiempo máximo que el sistema puede estar caído antes de restaurar el servicio.
- *Justificación:* El ecosistema creativo requiere disponibilidad continua. 2 horas es el tiempo estimado para aprovisionar un nuevo servidor cloud, descargar el Base Backup desde AWS S3 (~10GB estimados) y aplicar la cadena de archivos `WAL`.

I-E Riesgos, Incidentes y Acciones Concretas (Matriz de Mitigación)

La siguiente matriz resume los principales riesgos identificados y las acciones concretas para mitigarlos:

Cuadro I: Matriz de mitigación de riesgos

Riesgo / Incidente	Probabilidad	Impacto	Acción Concreta de Recuperación
Error Humano (DELETE/DROP): Un DBA o script malicioso borra la tabla pedidos o elimina registros sin WHERE.	Media	Alto	Ejecutar una recuperación lógica (Point-in-Time Recovery - <i>PITR</i>) en un servidor secundario (Staging) apuntando al segundo exacto antes del error, extraer la tabla afectada con <code>pg_dump</code> y restaurarla en el servidor de Producción.
Fallo Físico de Hardware: El disco del servidor PostgreSQL sufre daño irreparable.	Baja	Crítico	Despliegue de contingencia: Iniciar una instancia de repuesto. Descargar el último Full Backup desde el almacenamiento externo (S3), restaurarlo y aplicar los logs <i>WAL</i> acumulados hasta el último minuto funcional.
Corrupción de Datos a nivel de bloque: Un corte de energía abrupto daña los archivos de datos físicos de la base de datos.	Media	Alto	Apagar el clúster dañado. Reemplazar el directorio de datos (PGDATA) con el respaldo físico (<code>pg_basebackup</code>) más reciente y permitir que PostgreSQL realice un Crash Recovery usando los archivos <i>WAL</i> archivados.

II Diseño y justificación del plan semanal de respaldos

Para satisfacer el *RPO* de 15 minutos y proteger la infraestructura sin saturar los recursos del servidor durante horas pico, se ha diseñado una política híbrida de respaldos físicos y lógicos, combinando copias completas semanales con archivado continuo de logs de transacciones.

II-A Matriz de Planificación de Respaldos (7 Días)

Cuadro II: Planificación semanal de respaldos para ARTISYNC

Día	Hora	Tipo de Respaldo	Herramienta	Ubicación	Responsable	Retención
Domingo	02:00 AM	Full Backup Físico (Base Backup)	pg_basebackup	AWS S3 (Bucket Inmutable)	DBA (Bryan F.)	4 Semanas (GFS)
Lunes	03:00 AM	Diferencial (Logs consolidados)	pg_waldump / Script	Cloudflare R2 / Servidor NAS	DBA (Johan C.)	7 Días
Martes	03:00 AM	Diferencial (Logs consolidados)	pg_waldump / Script	Cloudflare R2 / Servidor NAS	DBA (Bryan F.)	7 Días
Miércoles	03:00 AM	Diferencial (Logs consolidados)	pg_waldump / Script	Cloudflare R2 / Servidor NAS	DBA (Johan C.)	7 Días
Jueves	03:00 AM	Diferencial (Logs consolidados)	pg_waldump / Script	Cloudflare R2 / Servidor NAS	DBA (Bryan F.)	7 Días
Viernes	03:00 AM	Diferencial (Logs consolidados)	pg_waldump / Script	Cloudflare R2 / Servidor NAS	DBA (Johan C.)	7 Días
Sábado	03:00 AM	Diferencial (Logs consolidados)	pg_waldump / Script	Cloudflare R2 / Servidor NAS	DBA (Bryan F.)	7 Días

Continúa en la siguiente página

Cuadro II – Continuación

Día	Hora	Tipo de Respaldo	Herramienta	Ubicación	Responsable	Retención
Lunes a Dom.	Continuo	Incremental (Archivado WAL)	archive_command	Servidor Externo dedicado (NFS)	Sistema Automatizado	14 Días
Sábado	10:00 AM	Simulacro de Restauración	Restore Script	Entorno Staging Local	Niurca B. / Jhon R.	N/A

II-B Justificación Técnica del Plan

- **Regla de Almacenamiento (3-2-1):** El plan cumple este estándar internacional. Se tienen 3 copias de los datos (Producción, Servidor NFS Local, y AWS S3). Se utilizan 2 medios distintos (Discos NVMe locales y Object Storage en la nube). Se tiene 1 copia totalmente externa (."off-site") en AWS S3 para desastres geográficos.
- **Frecuencia y Horarios:** El Full Backup se realiza el domingo a las 02:00 AM. Según el análisis de la plataforma ARTISYNC, la madrugada del domingo presenta el menor tráfico de usuarios y cierres de contratos, por lo que el impacto en el I/O del disco es mínimo.
- **Optimización de Espacio:** En PostgreSQL, un respaldo completo diario ocuparía demasiado espacio y ancho de banda. Al combinar un Base Backup semanal con un archivado continuo de WAL, el crecimiento del almacenamiento es gradual y predecible (solo se guarda el delta de cambios).
- **Pruebas de Restauración (Simulacros):** Un respaldo no sirve si no se puede restaurar. Se establece el sábado a las 10:00 AM como ventana obligatoria para que el equipo (Niurca y Jhon) levante una base de datos en blanco y valide que los archivos del último backup no estén corruptos.

III Demostración práctica en pgAdmin y procedimiento de recuperación

Esta sección documenta el paso a paso exacto que se ejecutará en la demostración del video para la recuperación lógica mediante la herramienta gráfica **pgAdmin 4**.

III-A Fase 1: Verificación de la base original

1. Abrir pgAdmin 4 y conectar al servidor de base de datos PostgreSQL.
2. Desplegar el árbol de servidores → Databases → Seleccionar la base `artisync_db`.
3. Desplegar Schemas → public → Tables para mostrar las 42 tablas del sistema.
4. Abrir el "Query Tool" ejecutar consultas de validación:

```
1 -- Verificar conteo de contratos
2 SELECT count(*) FROM contratos; -- Anotar el numero (ej. 150)
3
4 -- Verificar usuarios registrados
5 SELECT * FROM usuarios LIMIT 5;
```

Listing 1: Consultas de verificación inicial

III-B Fase 2: Creación del Respaldo Completo (Logical Backup)

1. Hacer clic derecho sobre la base de datos `artisync_db` y seleccionar **Backup**.
2. En la pestaña **General**:
 - *Filename*: Especificar la ruta de destino (ej. `C:\backups\artisync_full_dump.custom`).
 - *Format*: Seleccionar **Custom** (Vital: el formato custom viene comprimido y permite elegir qué restaurar después).
 - *Role*: `postgres` (o el usuario administrador).
 - *Encoding*: `UTF8`.
3. En la pestaña **Dump options**:
 - En "Type of objects", asegurar que esté en **"Pre-data, Data, Post-data"** (para que guarde la estructura, los datos, y los índices/triggers).
4. Hacer clic en **Backup**. Aparecerá una notificación en la parte inferior derecha: "Backup completed".
5. **Validación**: Mostrar en el explorador de archivos que el archivo `artisync_full_dump.custom` se ha generado, mostrar la fecha de modificación y su tamaño.

III-C Fase 3: Simulación de Desastre (El Error Humano)

1. Regresar al Query Tool en pgAdmin.
2. Ejecutar el siguiente comando destructivo para simular la pérdida total del esquema:

```
1 -- !!! ADVERTENCIA: Este comando es destructivo y solo debe ejecutarse en entornos de prueba !!!
2 DROP SCHEMA public CASCADE;
3 CREATE SCHEMA public;
```

Listing 2: Simulación de desastre

3. Ejecutar `SELECT * FROM contratos;`. El sistema retornará un error inmediato: **relation 'contratos' does not exist**", confirmando que la base de datos está vacía.

III-D Fase 4: Procedimiento de Restauración

1. Hacer clic derecho sobre la base de datos `artisync_db` (que ahora está vacía) y seleccionar **Restore**.
2. En la pestaña **General**:
 - *Format*: Custom or tar.
 - *Filename*: Buscar y seleccionar `C:\backups\artisync_full_dump.custom`.
 - *Role*: postgres.
3. En la pestaña **Restore options**:
 - Marcar la opción **"Clean before restore"** (No/Sí). (En este caso, como vaciamos el esquema, no es estrictamente necesario, pero es buena práctica para evitar conflictos de Primary Keys si quedaban restos).
4. Hacer clic en **Restore**.
5. **Validación final**: Abrir el Query Tool nuevamente.

```
1 -- Verificar que los datos fueron restaurados correctamente
2 SELECT count(*) FROM contratos; -- El resultado debe ser exactamente el mismo numero anotado en
   la Fase 1 (ej. 150)
```

Listing 3: Verificación post-restauración

IV Investigación y simulación Incremental/Diferencial (PostgreSQL WAL/PITR)

IV-A Fundamentación Teórica

- **Respaldo Completo (Full):** Copia íntegra de todos los datos en un momento dado. Es pesado y lento de generar, pero es el más fácil y rápido de restaurar.
- **Respaldo Diferencial:** Guarda los cambios realizados desde el último Respaldo Completo. Cada día el archivo es más grande. Para restaurar se necesita: El Full + El Diferencial del día específico.
- **Respaldo Incremental:** Guarda solo los cambios desde el último respaldo (sea full o incremental). Ocupa muy poco espacio. Para restaurar se necesita: El Full + Toda la cadena secuencial de incrementales en orden estricto.

IV-A1 La arquitectura de PostgreSQL y los WAL

A diferencia de motores como Oracle, PostgreSQL no tiene un comando nativo como `pg_dump --incremental`. La verdadera recuperación incremental en Postgres se realiza a nivel físico utilizando **WAL (Write-Ahead Log)**.

Un WAL es un archivo (usualmente de 16MB) donde Postgres escribe cada transacción antes de aplicarla a la tabla real. Al guardar el historial de todos estos archivos (Archivado Continuo) junto con un Respaldo Base (`pg_basebackup`), podemos lograr **PITR (Point-In-Time Recovery)**. Esto permite reproducir la historia de la base de datos hasta un milisegundo exacto (ej. detener la recuperación justo antes de un comando `DROP TABLE`).

IV-B Simulación de Cadena Incremental y PITR (Consola)

Para la demostración de recuperación avanzada, se utilizará la línea de comandos (Bash/CMD).

IV-B1 Paso 1: Configuración del Motor (Prerrequisitos)

Se debe demostrar que el archivo `postgresql.conf` tiene habilitado el archivado:

```
1 wal_level = replica
2 archive_mode = on
3 archive_command = 'cp %p /ruta/al/archivo/wal_archive/%f'
```

Listing 4: Configuración de archivado WAL en postgresql.conf

IV-B2 Paso 2: Generar el Respaldo Base (Full Físico)

Este es el "punto 0":

```
1 pg_basebackup -h localhost -D /var/lib/pgsqli/backups/base_artisync -U postgres -Ft -z -P
```

Listing 5: Generación del backup base

IV-B3 Paso 3: Generar la cadena Incremental (Nuevos datos)

Entrar a la base y generar nuevos registros que no están en el backup base:

```
1 INSERT INTO mensajes (id_sala, id_remitente, cuerpo_mensaje)
2 VALUES (1, 2, 'Mensaje critico post-backup');
3
4 -- Forzar el cierre del archivo WAL para que se envíe al archivo (simula el paso del tiempo)
```

```
5 SELECT pg_switch_wal();
```

Listing 6: Generación de datos incrementales

IV-B4 Paso 4: Simular Destrucción de la base física

```
1 sudo systemctl stop postgresql
2 rm -rf /var/lib/pgsql/data/*
```

Listing 7: Simulación de desastre físico

IV-B5 Paso 5: Recuperación Diferencial/Incremental (PITR)

1. Descomprimir el backup base en el directorio de datos vacío:

```
1 tar -xvf /var/lib/pgsql/backups/base_artisync/base.tar.gz -C /var/lib/pgsql/data/
```

Listing 8: Restauración del backup base

2. Crear el archivo de señal para indicar a Postgres que entre en modo recuperación:

```
1 touch /var/lib/pgsql/data/recovery.signal
```

Listing 9: Activación del modo recuperación

3. Configurar cómo debe leer la cadena incremental (los WALs) en `postgresql.conf` (o `recovery.conf` en versiones antiguas):

```
1 restore_command = 'cp /ruta/al/archivo/wal_archive/%f %p'
2 recovery_target_time = '2026-07-28 14:00:00' -- (Opcional: Si queremos parar en un momento exacto
   )
```

Listing 10: Configuración de recuperación PITR

4. Iniciar el servicio:

```
1 sudo systemctl start postgresql
```

Listing 11: Inicio del servicio

5. **Validación:** Entrar a la base y hacer `SELECT * FROM mensajes;`. El mensaje "Mensaje critico post-backup." estará ahí, demostrando que los archivos WAL aplicaron la cadena incremental exitosamente sobre el backup base.

V Conclusiones y Recomendaciones

V-A Conclusiones

El plan de respaldos y recuperación diseñado para ARTISYNC establece una base sólida para garantizar la continuidad del negocio y la integridad de los datos críticos de la plataforma. Las conclusiones principales son:

1. **Cumplimiento de objetivos:** Los *RPO* de 15 minutos y *RTO* de 2 horas son alcanzables mediante la combinación de respaldos completos semanales y archivado continuo de *WAL*, siempre que se mantenga la infraestructura adecuada.
2. **Protección de datos financieros:** La implementación del patrón Escrow requiere un enfoque especial en la integridad de las tablas `pagos_garantia` y `transacciones_pago`, lo cual se logra mediante el *PITR*.
3. **Pruebas periódicas:** Los simulacros de restauración son fundamentales para validar la efectividad del plan. Un respaldo sin capacidad de restauración es inútil.
4. **Automatización:** La automatización del archivado *WAL* y de los respaldos diferenciales reduce el error humano y garantiza la consistencia de las copias.

V-B Recomendaciones

Para fortalecer aún más la estrategia de continuidad del negocio de ARTISYNC, se recomienda:

1. **Implementar replicación en tiempo real (Streaming Replication):** Configurar un servidor réplica en caliente (hot standby) para reducir el *RTO* a menos de 30 minutos, permitiendo failover automático.
2. **Monitoreo proactivo:** Implementar un sistema de monitoreo que alerta sobre fallos en el archivado *WAL*, corrupción de backups o problemas de almacenamiento.
3. **Documentación y capacitación:** Mantener la documentación actualizada y capacitar a todos los miembros del equipo en los procedimientos de recuperación, especialmente en escenarios de desastre.
4. **Pruebas de carga:** Realizar pruebas de restauración con el volumen completo de datos (incluyendo la tabla `mensajes` con millones de registros) para validar los tiempos de recuperación.
5. **Retención de backups:** Considerar un esquema de retención de 6 meses para backups mensuales, almacenados en un medio de almacenamiento frío (Glacier o Deep Archive) para cumplir con requisitos legales.

Apéndice

A Script de Backup Automático (bash)

```
1 #!/bin/bash
2 # backup_artisync.sh - Script de respaldo automatico para ARTISYNC
3 # Uso: ./backup_artisync.sh [full|differential|wal]
4
5 set -e
6
7 # Variables de configuracion
8 BACKUP_DIR="/var/backups/artisync"
```

```

9 WAL_ARCHIVE="/var/lib/pgsql/wal_archive"
10 S3_BUCKET="s3://artisync-backups"
11 PG_USER="postgres"
12 PG_HOST="localhost"
13 PG_DB="artisync_db"
14
15 # Crear directorios si no existen
16 mkdir -p $BACKUP_DIR
17 mkdir -p $WAL_ARCHIVE
18
19 # Funcion para backup completo
20 do_full_backup() {
21     TIMESTAMP=$(date +%Y%m%d_%H%M%S)
22     echo "[$(date)] Iniciando backup completo..."
23
24     pg_basebackup -h $PG_HOST -D $BACKUP_DIR/base_$TIMESTAMP -U $PG_USER -Ft -z -P
25
26     # Sincronizar con S3
27     aws s3 sync $BACKUP_DIR/ $S3_BUCKET/ --exclude "*.tmp"
28
29     echo "[$(date)] Backup completo finalizado: $BACKUP_DIR/base_$TIMESTAMP"
30 }
31
32 # Funcion para forzar archivado WAL
33 do_wal_archive() {
34     echo "[$(date)] Forzando archivado WAL..."
35     psql -h $PG_HOST -U $PG_USER -d $PG_DB -c "SELECT pg_switch_wal();"
36
37     # Sincronizar WALs con S3
38     aws s3 sync $WAL_ARCHIVE/ $S3_BUCKET/wal/ --exclude "*.tmp"
39
40     echo "[$(date)] WAL archivado completado"
41 }
42
43 # Logica principal
44 case "$1" in
45     full)
46         do_full_backup
47         ;;
48     wal)
49         do_wal_archive
50         ;;
51     *)
52         echo "Uso: $0 {full|wal}"
53         exit 1
54         ;;
55 esac
56

```

```
57 exit 0
```

Listing 12: Script de backup automatizado para ARTISYNC

B Programación en cron

```
1 # Backup full semanal (Domingo 2:00 AM)
2 0 2 * * 0 /usr/local/bin/backup_artisync.sh full >> /var/log/backup_artisync.log 2>&1
3
4 # Archivado WAL cada 15 minutos
5 */15 * * * * /usr/local/bin/backup_artisync.sh wal >> /var/log/backup_artisync.log 2>&1
6
7 # Simulacro de restauracion (Sabado 10:00 AM)
8 0 10 * * 6 /usr/local/bin/restore_simulator.sh >> /var/log/restore_simulator.log 2>&1
```

Listing 13: Crontab para automatización

Referencias

- [1] S. Coello Quezada, O. Peñafiel Chele, and Á. Yanza Montalván, “Desarrollo de una aplicación web e-commerce B2C mediante tecnologías de desarrollo front-end y back-end. Caso de estudio ferretería ‘El Descanso’,” *Investigación, Tecnología e Innovación*, vol. 15, no. 19, Jul. 2023, doi: 10.53591/ITI.V15I19.1525.
- [2] R. Peres, M. Schreier, D. A. Schweidel, and A. Sorescu, “The creator economy: An introduction and a call for scholarly research,” *International Journal of Research in Marketing*, vol. 41, no. 3, pp. 403-410, Sep. 2024, doi: 10.1016/j.ijresmar.2024.07.005.
- [3] L. Gussek, A. Grabbe, and M. Wiesche, “Challenges of IT freelancers on digital labor platforms: A topic model approach,” *Electronic Markets*, vol. 33, no. 1, pp. 55, Oct. 2023, doi: 10.1007/S12525-023-00675-Y.
- [4] W. Tafesse and M. Dayan, “Content creators’ participation in the creator economy: Examining the effect of creators’ content sharing frequency on user engagement behavior on digital platforms,” *Journal of Retailing and Consumer Services*, vol. 73, Jul. 2023, doi: 10.1016/J.JRETCONSER.2023.103357.
- [5] P. Monsalve-Obreque, P. Vargas-Villarroel, Y. Hormazabal-Astorga, J. Hochstetter-Diez, J. Bustos-Gómez, and M. Diéguez-Rebolledo, “Proposal to Improve the E-Commerce Platform Development Process with an Exploratory Case Study in Chile,” *Applied Sciences (Switzerland)*, vol. 13, no. 14, Jul. 2023, doi: 10.3390/APP13148362.
- [6] T. Wulfert and R. Schütte, “Retailer’s Dual Role in Digital Marketplaces,” *SN Computer Science*, vol. 3, no. 3, pp. 206, Mar. 2022, doi: 10.1007/S42979-022-01098-W.
- [7] PostgreSQL Global Development Group, “PostgreSQL 18 Documentation: Backup and Restore,” *PostgreSQL Official Documentation*, 2024. [Online]. Available: <https://www.postgresql.org/docs/18/backup.html>
- [8] PGBackRest Development Team, “PGBackRest: Reliable PostgreSQL Backup & Restore,” *PGBackRest Documentation*, 2024. [Online]. Available: <https://pgbackrest.org/>